PhD Literature Reviews

Bruno Costa Alves Freire

September 26, 2025

1 Introduction

This document is meant to assemble drafts of literature review notes.

1.1 Target literature

1.1.1 PhD topic specific reading list reading

- (**??**) [1] Modern Coding Theory
- (**??**) [2] Efficient Erasure Correcting Codes
- (2.1) [3] Quantum LDPC codes with positive rate and minimum distance proportional to $n^{1/2}$
- (2.2) [4] Decoding Across the Quantum LDPC Code Landscape
- () [5] Fast erasure decoder for hypergraph product codes
- (2.3) [6] Asymptotically Good Quantum and Locally Testable Classical LDPC Codes
- () [7] Quantum Tanner codes
- () [8] Quantum Expander Codes
- () [9] Combining hard and soft decoders for hypergraph product codes
- () [10] Efficient soft-output decoders for the surface code
- () [11] Weight Reduced Stabilizer Codes with Lower Overhead
- () [12] High-threshold, low-overhead and single-shot decodable fault-tolerant quantum memory
- () [13] Ambiguity Clustering: an accurate and efficient decoder for qLDPC codes
- () [14] On the iterative decoding of sparse quantum codes
- () [15] A topological theory for qLDPC: non-Clifford gates and magic state fountain on homological product codes with constant rate and beyond the $N^{1/3}$ distance barrier
- () [16] Asymptotically good CSS-T codes exist
- () [17] Computing Efficiently in QLDPC Codes
- () [18] A recursive approach to low complexity codes

1.1.2 General FTQC literacy

- () [19] Scheme for reducing decoherence in quantum computer memory
- () [20] Error Correcting Codes in Quantum Theory
- () [21] Stabilizer Codes and Quantum Error Correction
- () [22] Topological quantum memory
- () [23] Surface codes: Towards practical large-scale quantum computation
- () [24] Sparse Blossom: correcting a million errors per core second with minimum-weight matching
- () [25] Targeted Clifford logical gates for hypergraph product codes

1.1.3 Notes from NISQ2LSQ workshop in Marseille 22-24/01/2025

- Nicolas SAUSSAY : A variant of the Bravyi-Terhal bound for arbitrary boundary conditions. (**??** for notes);
- Wouter ROZENDAAL : An upper bound on the minimum distance of Abelian two-block group algebra codes. (**??** for notes);
- Thomas VAN HIMBEECK : Can we achieve the hashing bound with small weight quantum codes? (**??** for notes);
- Clément POIRSON : Engineering CSS surgery to perform logical free phase ZX-diagrams. (**??** for notes);
- Paul HILAIRE : Tailoring QEC codes to a photonic FTQC architecture. (**??** for notes);
- Mike VASMER : Single-shot and measurement-based quantum error correction via fault complexes. (**??** for notes);
- Samuel NOVAK : Basics of rotors: what is it, and why is it $\mathbb{Z}$? (**??** for notes);
- Christophe VUILLOT : Tiger codes. (**??** for notes);
- Gilles ZEMOR : Protecting Clifford circuits with rank-metric techniques. (**??** for notes);
- Virgile GUEMARD : Moderate length quantum Tanner codes. (**??** for notes);

1.1.4 On the decoding vs. decoder benchmarking for quantum erasure channels question

When actually decoding an erasure, in the classical case, one needs to output a codeword (or the completion of it) in function of the non-erased part of the received message. In the quantum case, the output should be a correction to be applied on the qubits, that must be compatible with a measured syndrome, and restricted to the support of the erasure. In both cases, if the decoder is not allowed to give up (signal failure), it is responsible for outputting something, that is, it should either be sure that there is a unique answer, or guess one among the equally likely possibilities. The point here is, for an actual decoder, this output depends (even if in a simple way) on something more than just the erased subset (the non-erased bits in the classical case, the measured syndromes in the quantum case). However, if we are interested solely in measuring the failure probability of the decoder (assuming it fails whenever it's not sure of the output), i.e., benchmarking it, we don't really need any information apart from the erasure subset.

This fact is simple enough in the classical case (cf. Richardson & Urbanke section 3.6.1). In the quantum case, let us provide a clearer argument as to why this is also true. Let $H_X, H_Z \in \mathbb{F}_2^{R\times N}$ define a CSS code $\mathcal{Q}$ (for simplicity, the same would work for any stabilizer code), let $\mathcal{E} \subset [N]$ be the erasure subset, and $E \leq \mathcal{E}$ is the error pattern resulting from the syndrome measurement (that is, if the initial state was $|\psi\rangle \in \mathcal{Q}$, the state after the syndrome measurement is $E|\psi\rangle$). For simplicity, we'll refer to the X/Z component of E with

the same notation, meaning $E \in \mathbb{F}_2^N$ and $\operatorname{supp} E \subset \mathcal{E}$. Let $\sigma_{Z/X} = H_{Z/X} E$ denote the syndrome measured at the Z/X stabilizers (capturing the X/Z component of E, resp.).

Define $\mathcal{C}(\mathcal{E}, \sigma) := \{E \in \mathbb{F}_2^N\big|_{\mathcal{E}} : H_Z\big|_{\mathcal{E}} E = \sigma\}$ as the set of corrections that are compatible with the syndrome σ and supported inside the erasure $\mathcal{E}$. Notice that $\mathcal{C}(\mathcal{E}, \sigma)$ is a vector subspace of $\mathbb{F}_2^N\big|_{\mathcal{E}}$. Moreover, let $\mathcal{S}_X = \operatorname{im} H_X^T = \ker \eta(H_X^T) \subset \mathbb{F}_2^N$ denote the group of X-type stabilizers, also seen as a subspace of $\mathbb{F}_2^N$, and its restriction $\mathcal{S}_X\big|_{\mathcal{E}} := \{S\big|_{\mathcal{E}} : S \in \mathcal{S}_X, \operatorname{supp} S \subset \mathcal{E}\} = \ker \eta(H_X^T)\big|_{\mathcal{E}}$ formed by the stabilizers that are supported in the erasure.

The correctability condition of the erasure $\mathcal{E}$ is given by

$$\mathcal{E} \text{ is correctable} \iff \frac{\mathcal{C}(\mathcal{E}, \sigma)}{\mathcal{S}_X\big|_{\mathcal{E}}} \cong \{\overline{0}\},$$

that is, when we quotient the subspace of compatible corrections by the stabilizer group, we should have only one equivalence class, otherwise, there will be logical errors hiding in the erasure. What we claim here is that, for all possible syndromes σ, we'll have $\#\frac{\mathcal{C}(\mathcal{E}, \sigma)}{\mathcal{S}_X\big|_{\mathcal{E}}} = \#\frac{\mathcal{C}(\mathcal{E}, 0)}{\mathcal{S}_X\big|_{\mathcal{E}}}$, which will in turn imply that the correctability condition is independent of the particular syndrome.

To prove this, we'll first assume that the syndrome σ is caused by an error supported in the erasure, i.e., $\exists E \in \mathbb{F}_2^N$ s.t. $H_Z E = \sigma$ and $\operatorname{supp} E \subset \mathcal{E}$. That implies that $\mathcal{C}(\mathcal{E}, \sigma) \neq \varnothing$. Thus, let $\overline{C} \in \frac{\mathcal{C}(\mathcal{E}, \sigma)}{\mathcal{S}_X\big|_{\mathcal{E}}}$, meaning $\overline{C} = C + \mathcal{S}_X\big|_{\mathcal{E}}$, $C \leq \mathcal{E}$ and $H_Z\big|_{\mathcal{E}} C = \sigma$. It is then easy to see that $\frac{\mathcal{C}(\mathcal{E}, \sigma)}{\mathcal{S}_X\big|_{\mathcal{E}}} = \frac{\mathcal{C}(\mathcal{E}, 0)}{\mathcal{S}_X\big|_{\mathcal{E}}} + \overline{C}$.

That proves that the correctability condition is independent on the syndrome, which means that the performance of the maximum likelihood decoder can be assessed with random erasure samples assuming only trivial syndrome. The same can be said for other decoders such as peeling and its variants, to the extend that their progression is not impacted by the particular value of the syndrome. In the following, let us analyze in more detail the operation of peeling, pruned peeling and branching to further convince ourselves of this fact.

Analyze the progression of peeling

1.1.5 Conjectures about the capabilities of branching vs. peeling and ML

1. At the asymptotic regime ($p \to 0$), the branching algorithm can match ML's performance if the number of available guesses g is at least $d - s$, where d is the code distance and s is the stopping distance.

Handwavy argument: For an erasure rate of p, the logical error rate is given by $\sum_{\mathcal{E} \subset [N]} p^{|\mathcal{E}|}(1-p)^{N-|\mathcal{E}|} \cdot [\mathcal{E} \text{ leads to a decoding failure}]$. Consider $\mathcal{E}$ such that $|\mathcal{E}| < d$. That implies that, for this instance of erasure error, the ML algorithm would succeed, i.e., $\mathcal{E}$ cannot contain a nontrivial logical operator. Under this assumption, the branching algorithm should only fail if it runs out of guesses before peeling $\mathcal{E}$ completely. Now, we know that for any erasure $\mathcal{E}'$ with $|\mathcal{E}'| < s$ (the stopping distance), the peeling algorithm is able to completely peel off $\mathcal{E}'$ and thus succeed. Since we assume that branching has at least $g = d - s$ guesses to spare, and it starts with an erasure pattern $\mathcal{E}$ of size $\mathcal{E} < d$, in the worst-case scenario peeling will block and trigger the branching mechanism as long as we have guesses left. However, after $d - s$ guesses, the size of the residual erasure will be less than $d - (d - s) = s$, and so it cannot still be a stopping set. That is, after at most $d - s$ guesses, the residual erasure gets too small to stop peeling from proceeding until the end. Thus, we get to the end with some number of branches (which may or may not have been cut along the execution), which we know must be all valid (that is, syndrome compliant and contained in the original erasure) and stabilizer-equivalent. To see why the remaining branches must be valid, just recall that branching has a consistency-check mechanism every time if finds a parity-check equation that had become dangling but whose dangling bit has been unerased before it was evaluated. Whenever these so-called empty equations are evaluated, they catch any branches that cause an inconsistency for it and trigger their cutting. Finally, to see why all the branches must be stabilizer-equivalent, just recall that, since $|\mathcal{E}| < d$, it cannot contain a nontrivial logical operator, and thus, all syndrome and erasure-compliant corrections must be stabilizer-equivalent. This way, we have that, for all $\mathcal{E}$ such that $|\mathcal{E}| < d$, $[\mathcal{E} \text{ leads to a decoding failure}] = 0$, and thus

$$p_L = \sum_{|\mathcal{E}|\geq d} p^{|\mathcal{E}|}(1-p)^{N-|\mathcal{E}|} \cdot [\mathcal{E} \text{ leads to a decoding failure}] = O(p^d).$$

Now, notice that the above argument gives us the same decay exponent d as for the ML algorithm, but we can't say the same about the constant, that is, how many erasure patterns $\mathcal{E}$ of size d will lead to a decoding failure for branching? Certainly at least as many as for ML, but could there be erasure patterns of size d for which ML succeeds and branching still fails even with $g = d - s$ guesses? Possibly yes. Nothing rules out the existence of d-sized erasure patterns that don't cover logical operators, but that contain s-sized stopping sets. To get rid of those, it suffices to add just one more guess, so $g \geq d - s + 1$ would be enough to match ML's asymptotic performance in both exponent and constant factor. It's good to highlight that it does not mean that the logical error rate for both algorithms will be exactly the same, because we can still have very large erasure patterns for which ML succeeds, but that block peeling sufficiently many times to exhaust all guesses lead to branching failure. However, those large patterns will have a negligible contribution to the logical error rate as $p \to 0$, in comparison to the others.

1.1.6 On the difference between *in silico* simulations vs. experimental determinations of logical error rates

So far, whenever I did simulations of the performance of codes, be it for the erasure channel or Pauli channels, for whatever decoder, the overall pipeline was simple: first, I sample some error pattern (Pauli string or erasure subset), compute the syndromes from it (I always assumed perfect measurements, but I could also include measurement error without changing much the pipeline), decode the syndrome/erasure (if solely for benchmarking purposes, I could just check whether the decoder is successful or not, not necessarily computing its output) and finally checking whether the predicted error/correction successfully brings the system back to the correct codestate. In all this procedure, not only are the simulations independent of the initial codestate (they don't affect any aspect of the decoding and assessment of correctness of the decoder's output), but they are also independent of data qubit measurements. The problem is, by keeping track of the physical error that affects the data (and eventually also the ancilla) qubits, I was doing something that is not compatible with an experimental setup. In experiments, we can only have access to information that we measure, and thus the whole benchmarking pipeline I described is impossible.

How then can we measure the performance of a code/decoder? How can we tell whether the decoder's correction brings us back to the initial codestate? In the *in silico* simulations, the criterion was basically that the *residual error* (that is, physical error minus decoder's correction) should be a stabilizer. That means we should make sure to return to the codespace (which is guaranteed in the absence of measurement error), and furthermore return to the same codestate as before. Well, the residual error is not accessible in the experimental setup. Moreover, since the measurements are noisy, and assuming that the decoder operates only with input from ancilla measurements, we don't have a guarantee of returning to the codespace. For the purpose of benchmarking the code/decoder in a memory experiment, we can allow ourselves to measure the data qubits at the end (but not feed them to the decoder), so we need to somehow extract (measure) information about this residual error from the data qubits.

It is easy to see that, if the residual error is not a stabilizer, it may flip some of the stabilizers (in which case it takes us out of the codespace), or, if it doesn't, then it is a (non trivial) logical operator, in which case it will flip some other logical operators, e.g., the Z_L logical operator is flipped by X_L or Y_L, assuming each of those act on the same logical qubit. If we assume that our logical qubit register is analogous to a physical register, and that each logical qubit undergoes, independently (or not?) some error channel (let's consider only Pauli channels at first), we could model the logical error channel as a Kraus map

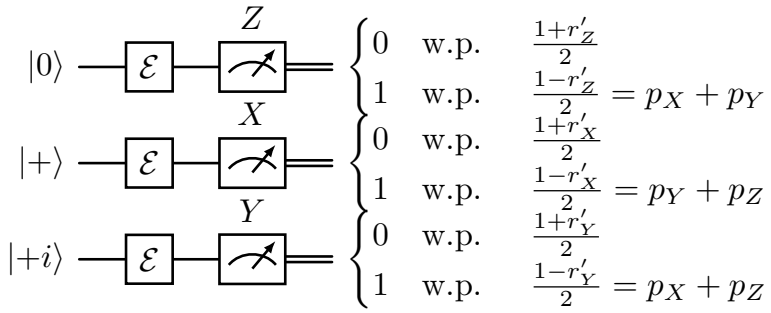
$$\mathcal{E}_L : \rho \mapsto (1-p_L)\rho + \sum_{W\in\{X,Y,Z\}} p_{W,L} W_L \rho W_L,$$

and set out experiments to estimate the parameters $p_{W,L}$. Note, however, that the W_L operators are only well defined if we assume that $\rho = |\psi\rangle\langle\psi|$ is in the codespace, because they are in fact *cosets* of operators by the stabilizer group. Should we include some probability of getting out of the codespace? In any case, how do we go about estimating those parameters?

In the physical setting, what we do is quantum process tomography. If we had a physical Kraus map of the form $\mathcal{E}(\rho) = (1-p)\rho + p_X X\rho X + p_Y Y\rho Y + p_Z Z\rho Z$ (where $p = p_X + p_Y + p_Z$), we could estimate its parameters by looking at its action on a density operator $\rho = \frac{1}{2}(I + \vec{r}\cdot\vec{\sigma})$,

$$\mathcal{E}(\rho) = \frac{I + [(1-p) + p_X - p_Y - p_Z]r_X X + [\dots]}{2} = \frac{I + \sum_{W\in\{X,Y,Z\}}[1-2(p-p_W)]r_W W}{2}.$$

Define $s_W = 1 - 2(p - p_W)$ as the *shrinking factor* in the W axis, $W \in \{X, Y, Z\}$. The resulting density operator $\rho' = \mathcal{E}(\rho)$ is given by $\frac{1}{2}(I + \vec{r'}\cdot\vec{\sigma})$, where $\vec{r'}$ describes the position of the physical qubits in the Bloch sphere after undergoing the error channel $\mathcal{E}$. By preparing ρ in different initial states and measuring ρ' in different bases, we can estimate the shrinking factors s_W, and thus the Pauli error parameters p_W, with the following circuits:



By preparing the initial states so that we have $r_W = 1$, we measure 1 (or -1 in the $+1/-1$ convention) in the corresponding basis with probability $\frac{1-s_W}{2}$, which will give us a linear system that can be further solved for p_W.

We can think of doing a similar thing for logical qubits. Initialize the register in the logical $|0\rangle_L$, $|+\rangle_L$ or $|+i\rangle_L$ states, run some cycles of syndrome extraction followed by the application of the decoder's correction, and then measure the logical Z_L, X_L or Y_L operators. There are some problems here though. The logical operators have multiple representatives, and the measurements of those will not disagree if we are not in the codespace.

Questions:

- When benchmarking codes/decoders experimentally, should we feed the decoder with data qubit measurements?
- Assuming the decoder can't use data qubit measurements to compute the correction, we have no guarantee of returning to the codespace. Thus, different representatives of logical operators will disagree upon measurement. What should we measure then?
- For general stabilizer codes, how do we go about measuring a logical observable from the data qubits? How many of such observables can we measure simultaneously (i.e., in the same shot)? Certainly at most k (= # of logical qubits) since there are pairs of logical operators that anti-commute.
- How do we compatibilize the definition of logical error rate from the *in silico* computations with the QPT approach described here? Keeping in mind that in the former definition, any nontrivial logical operator yields a logical error, but depending on the initial codestate, some nontrivial logical operators leave the state unchanged (e.g., if we are in the logical $|0\rangle_L$ state, the logical Z_L operator doesn't amount to a logical error). That is, what the *in silico* simulations compute is akin to $p_L = p_{X,L} + p_{Y,L} + p_{Z,L}$. What should we use as a figure of merit then?

2 Short analyses and comments

2.1 HGP codes

Key paper by Tillich and Zémor, introduced the HGP code construction and changed the state-of-the-art of QEC codes in terms of the asymptotics of the minimum distance parameter (previous SOTA was $d = O(\log n)$, upgraded to $d = \Theta(\sqrt{n})$), for qLDPC codes with fixed rate (i.e., $k = \Theta(n)$). The paper presents the HGP construction and draws connections with related concepts, such as hypergraphs, explains how the HGP codes generalize toric codes, and proves the asymptotic properties of HGP codes. The main advantage of this construction is it's capability of creating CSS codes from any two classical codes, while preserving the LDPC property.

2.1.1 Further reflections and remarks

During the internship we overlooked a problem related to the code with parameters $[\![1225, 65]\!]$. The classical code used to produce this HGP has $m = 21$ equations and $n = 28$ bits, but it's parity-check matrix has rank only 20, so the transpose code is nontrivial, which is why we can even get an HGP code whose dimension is not a perfect square to begin with. The problem is, as we deform the code to traverse the state space, the rank of the parity-check matrix may increase, leading to the reduction of the resulting HGP code's dimension. The lowest dimension we can get is for when the classical parity-check matrix is full-rank, which is the case for the other three families, hence why we overlooked this problem during our investigations and didn't bother checking for a decrease of the codes' dimension.

This sheds light on the fact that the HGP construction *is not a function of the two classical codes*, but rather of their *parity-check matrix / Tanner graph representations*. The same pair of classical codes can have different matrix and graphical representations, which yield effectively different HGP codes. Adding or suppressing redundant parity-check equations (rows in the matrix) doesn't change the underlying code, but adds or suppresses check nodes in the Tanner graph, all of which will directly reflect on number of physical qubits of the HGP code. Moreover, replacing equations by linear combinations of the others implies changing the connectivity structure of the Tanner graph, going as far as changing the number of edges and the degree distributions, so the LDPC property is also perturbed by this kind of change. From another point of view, the HGP construction depends on the transpose codes (or at least their matrices and graphs), so applying row equivalent transformations to H translates into column equivalent transformations to H^T, which don't necessarily preserve the transpose code. So far, the only identified transformations that can be applied to matrices or graphs while preserving in some sense the HGP code are row/column permutations or graph isomorphisms. I wonder if the same could be said about any isometric isomorphisms (i.e., any invertible linear transformation that preserves the Hamming distance) of the classical code.

In sync with the discussion on **??**, the choice of the specific matrix or graph representation of the code may be irrelevant for ML decoding, but it becomes relevant for iterative decoding algorithms (possibly also for encoding?). For the HGP construction, regardless of the decoding approach, the code itself is different according to the matrix/graph representation choice of the classical codes.

2.2 Decoding qLDPC codes

This paper proposes a decoding strategy for HGP codes based on a combination of Belief Propagation (BP) and Order Statistics decoding (OSD). There is a github repo with the code available.

2.3 Lifted product codes

Important paper by Panteleev and Kalachev, first to introduce asymptotically good quantum LDPC codes. Asymptotically good means that the codes have both linear dimension and minimum distance, $[\![n, k = \Theta(n), d = \Theta(n)]\!]$. The construction is based on an operation from homological algebra known as lifting, hence the name lifted product codes.

2.4

3 In-depth reading

4 Material for peeling+guessing paper

Key points:

1. Adaptation of the Maxwell decoder to the quantum CSS case
2. Introduce the guessing mechanism
3. Introduce pruning and explain its role
4. Theoretical claims about performances: g ¿= d-s and g ¿= d-s+1
5. Complexity analysis of peeling + guessing O(N * 2**g)
    - discuss the O(n**2) post-processing that occurs only in benchmarking, not on operation
6. Applications (simulation results)
    - optimized HGP codes
    - BB codes
7. Non CSS generalization
8. Pauli error?
9. Comparison with existing methods
    - BPGD for erasures (Gokduman, Hanwen, Pfister)
    - Clustering (Pfister)
    - Degenerate erasure decoding (Kao-Yueh Kuo)
10. Rare events simulation? To go further down the p-¿0 regime

priority: 9 ¿ 6 ¿ 10
Paper outline:

4.1 Introduction

In quantum computing, quantum error correction techniques are essential to render quantum systems sufficiently robust to noise and capable of performing useful digital algorithms. Depending on the physical platform, different error models may more accurately describe the noise affecting the system. One particularly simplified and widely studied model is the quantum erasure channel, in which qubits are affected by errors at known locations; that is, each qubit can be erased with some probability independently, and the positions of all erased qubits are known to the decoder. An erased qubit can be modeled as a fully depolarized, maximally mixed state, and the underlying error affecting an erased qubit corresponds to a random Pauli operator determined upon syndrome measurement. Although simplified, the erasure channel provides valuable insights for analyzing the performance of quantum codes and decoding algorithms, and can be adapted to more realistic noise models. Moreover, some physical platforms can transform their native errors into erasures, which greatly simplifies the task of syndrome decoding.

In both the classical and quantum settings, the erasure decoding problem admits an optimal solution in terms of finding the most likely error configuration. This maximum-likelihood solution can be obtained by solving a linear system over the binary field, with Gaussian elimination as the most efficient general-purpose method. However, its cubic complexity in the number of qubits becomes prohibitive for large systems. This has motivated the development of iterative decoding algorithms with lower computational complexity at the expense of some loss in decoding performance. Such methods include message-passing algorithms, which exist in various forms for different noise models and have also been applied in the quantum setting. For the erasure channel, message passing reduces to the well-known peeling decoder.

In the classical case, the peeling decoder operates by progressively resolving erased bits, treating them as unknown variables in the parity-check equations. Whenever an equation contains a single unknown variable,

it can be solved directly, revealing the corresponding bit. As the algorithm progresses, more equations become solvable, and the procedure continues until either all erasures are resolved or no single-unknown equations remain, at which point the algorithm halts with a failure. In the quantum setting, for CSS codes, the peeling decoder is applied independently to the X and Z components of the code using the same initial erasure pattern; successful decoding requires that both parts be peeled successfully.

Numerous improvements to peeling have been proposed in the classical coding literature. One notable variant is the "Peeling + Guessing" decoder, which addresses situations where peeling stalls. In this approach, when no single-unknown equations remain, the decoder guesses one of the remaining erased bits and proceeds as in standard peeling. Each guess introduces branching in the decoding process, corresponding to the possible bit values. As decoding progresses, some branches may be discarded when inconsistencies arise (for example, when a fully determined equation evaluates incorrectly). This approach can be viewed as a form of list decoding, producing all candidate solutions consistent with the observed syndrome. If a single solution remains, the erasure is deemed correctable; otherwise, the decoder may return multiple equally likely solutions or declare a failure if the number of guesses exceeds a predefined limit.

Building on this line of work, analogous ideas can be extended to the quantum erasure channel for CSS codes. In such a scenario, after running a peeling-plus-guessing procedure, the resulting list of candidate corrections can be further reduced by computing their equivalence classes modulo the stabilizer group. If a single equivalence class remains, the decoding succeeds; if multiple equivalence classes appear, this indicates that the erasure covers a logical operator and is therefore not correctable. This additional step accounts for the degeneracy inherent to quantum stabilizer codes.

4.2 Algorithm description

Algorithm 1 Peeling + Guessing Decoder

Input: Subset E of erased qubits (binary vector), Syndrome vector, Guessing budget G

Initialize $g \leftarrow 0$
while $|E| > 0$ **do**
  Search for a dangling check
  if there is a dangling check **then**
    Call **DanglingCheckResolution** subroutine
  else
    if $g < G$ **then**
      Call **Guessing** subroutine
    else
      signal failure
    end if
  end if
end while
while there are empty equations **do**
  Enforce consistency of empty equations (cut inconsistent branches)
end while
for all remaining branches **do**
  Compute equivalence class modulo stabilizers
end for
if all branches belong to the same class **then**

  return any branch
else
  signal failure
end if

4.3 Theoretical results

1. At the asymptotic regime ($p \to 0$), the branching algorithm can match ML's performance up to a constant factor if the number of available guesses g is at least $d - s$, where d is the code distance and s is the stopping distance. Moreover, if $g \geq d - s + 1$, the performance of the branching algorithm will be asymptotically equivalent to that of ML.

Handwavy argument: For an erasure rate of p, the logical error rate is given by $\sum_{\mathcal{E} \subset [N]} p^{|\mathcal{E}|}(1-p)^{N-|\mathcal{E}|} \cdot [\mathcal{E} \text{ leads to a decoding failure}]$. Consider $\mathcal{E}$ such that $|\mathcal{E}| < d$. That implies that, for this instance of erasure error, the ML algorithm would succeed, i.e., $\mathcal{E}$ cannot contain a nontrivial logical operator. Under this assumption, the branching algorithm should only fail if it runs out of guesses before peeling $\mathcal{E}$ completely. Now, we know that for any erasure $\mathcal{E}'$ with $|\mathcal{E}'| < s$ (the stopping distance), the peeling algorithm is able to completely peel off $\mathcal{E}'$ and thus succeed. Since we assume that branching has at least $g = d - s$ guesses to spare, and it starts with an erasure pattern $\mathcal{E}$ of size $\mathcal{E} < d$, in the worst-case scenario peeling will block and trigger the branching mechanism as long as we have guesses left. However, after $d - s$ guesses, the size of the residual erasure will be less than $d - (d - s) = s$, and so it cannot still be a stopping set. That is, after at most $d - s$ guesses, the residual erasure gets too small to stop peeling from proceeding until the end. Thus, we get to the end with some number of branches (which may or may not have been cut along the execution), which we know must be all valid (that is, syndrome compliant and contained in the original erasure) and stabilizer-equivalent. To see why the remaining branches must be valid, just recall that branching has a consistency-check mechanism every time if finds a parity-check equation that had become dangling but whose dangling bit has been unerased before it was evaluated. Whenever these so-called empty equations are evaluated, they catch any branches that cause an inconsistency for it and trigger their cutting. Finally, to see why all the branches must be stabilizer-equivalent, just recall that, since $|\mathcal{E}| < d$, it cannot contain a nontrivial logical operator, and thus, all syndrome and erasure-compliant corrections must be stabilizer-equivalent. This way, we have that, for all $\mathcal{E}$ such that $|\mathcal{E}| < d$, $[\mathcal{E} \text{ leads to a decoding failure}] = 0$, and thus

$$p_L = \sum_{|\mathcal{E}| \geq d} p^{|\mathcal{E}|}(1-p)^{N-|\mathcal{E}|} \cdot [\mathcal{E} \text{ leads to a decoding failure}] = O(p^d).$$

Now, notice that the above argument gives us the same decay exponent d as for the ML algorithm, but we can't say the same about the constant, that is, how many erasure patterns $\mathcal{E}$ of size d will lead to a decoding failure for branching? Certainly at least as many as for ML, but could there be erasure patterns of size d for which ML succeeds and branching still fails even with $g = d - s$ guesses? Possibly yes. Nothing rules out the existence of d-sized erasure patterns that don't cover logical operators, but that contain s-sized stopping sets. To get rid of those, it suffices to add just one more guess, so $g \geq d - s + 1$ would be enough to match ML's asymptotic performance in both exponent and constant factor. It's good to highlight that it does not mean that the logical error rate for both algorithms will be exactly the same, because we can still have very large erasure patterns for which ML succeeds, but that block peeling sufficiently many times to exhaust all guesses lead to branching failure. However, those large patterns will have a negligible contribution to the logical error rate as $p \to 0$, in comparison to the others.

4.4 Comparison with prior work

4.4.1 Erasure Decoding for QLDPC codes with BPGD

In [26], the authors adapt the BPGD algorithm to the case of erasure errors. Previously, the BPGD algorithm had been applied to bit-flip and depolarizing noise in the code capacity model, demonstrating excellent performance. In comparison to Connolly's decoders (pruned peeling and VH), BPGD outperforms pruned peeling and lags behind VH in terms of accuracy, however, it has much lower complexity than VH. It can be improved with damping techniques and by adjusting the prior LLRs. Moreover, it is naturally applicable to Pauli+erasure nose, due to dealing essentially with soft information.

But what is BPGD concretely? The algorithm builds on top of regular BP by employing Guided Decimation. The key issue that guided decimation tries to address is the convergence failure of BP due to the existence of small cycles in the Tanner graphs of QLDPC codes (it is the same issue that gives rise to small

stopping sets, which leads to failure in the peeling decoder). In some sense, it is an alternative approach (or even an adaptation to the message-passing context) to the pruning technique. Concretely, the algorithm consists on several rounds of running BP for a fixed number of iterations, and at the end of this step, if convergence is not achieved, you pick the most confident bit (the largest absolute value LLR) and round it to 0 or 1 (this is what they call decimation). Then proceed to the next round, where you run BP again with the remaining bits until convergence or until you have to do decimation again.

This plays a similar role as pruning, by picking a bit in the support of a stabilizer and fixing its correction to 0, effectively gauging the stabilizer in question. However, what guided decimation does is not error-proof, i.e., you can end up incorrectly decimating a bit and leading to a wrong solution in the end. In other words, you augment the probability of convergence for BP, but you introduce some probability of converging to a wrong solution. In comparison to pruned peeling, the GD mechanism allows the algorithm to effectively guess some bits along the way, but with no filter mechanisms for wrong guesses, apart from, eventually, guesses that correspond to fixing qubits in the support of a stabilizer. The performance though is greatly improved with respect to pruned peeling, showing that allowing for guesses regardless of stabilizer support is a powerful strategy, even in the absence of guess validation mechanisms. Note: perhaps this is a good way to proceed when we consider measurement errors.

Numerical details of the implementation that are worth mentioning is that, instead of initializing the LLRs of the non-erased/erased bits as $\pm\infty/0$, you pick finite values such as $\lambda_{\max} = 25$ and $\lambda_{\min} \approx 0$. Moreover, the "bonus" mechanisms they introduced, namely LLR adjusting and damping, play a role in improving the convergence rate of BPGD, which in turn improves the performance for some erasure rates. The LLR adjusting consists on initializing the non-erased qubits with smaller confidence (some fraction of the maximum LLR). The optimal fractions are tuned by hand depending on the erasure rate. Damping is the process of updating the messages that are passed along the edges by a sort of moving average, instead of simply replacing the previous value with the new one. Usually, they keep a small proportion of the old value (γ is close to 1). The smaller the damping factor λ, more damping you have. The intuition behind damping is that it helps counter oscillations and improves converge of BP.

Apparently, they too are restricted to CSS codes, because they can split the X and Z parts and solve two classical problems. Not that it's a demerit, after all no one cares about non-CSS codes. But there is indeed a concept of quaternary/Pauli BP, that is, BP on the alphabet $\{I, X, Y, Z\}$.

4.4.2 Cluster decomposition for improved erasure decoding of QLDPC codes

In [27], the authors propose a new erasure decoding algorithm that combines peeling with a cluster decomposition post-processing step, that works as a generalization of the VH cluster decomposition, and applies to any CSS code (not just HGP). The cluster decomposition can be tuned to allow arbitrary cluster sizes and thus achieve ML performance (with runtime lower than full Gaussian elimination), or restrict the size of the clusters (over which ML decoding is applied) below a constant size to obtain linear time decoding, all while keeping a competitive performance with VH and ML.

This cluster size as a tunable parameter plays a similar role to the number of guesses in our algorithm, i.e., by allowing arbitrarily many guesses, we achieve ML performance, but by limiting it to a constant amount, we achieve linear time decoding. Above a certain threshold, we can change the operation mode from branching to symbolic variables and basically mimic ML, but avoiding the exponential cost, sticking to at most cubic complexity.

In more detail, the algorithm consists on running regular peeling until a stopping set is reached, and then the residual Tanner graph is decomposed a forest of clusters, each cluster consisting of a biconnected component of such graph. Pruning can be employed before going to the cluster decomposition step, but they don't explore it here. In order to compute such decomposition, they employ the Hopcroft-Tarjan's algorithm, which finds biconnected components and articulation points of a graph, and has linear complexity on the number of edges and vertices. Next, they build a bipartite graph where one set of nodes corresponds to the clusters (biconnected components) and the other set corresponds to the articulation point. Edges between those occur when a cluster contains an articulation point. This naturally leads to an acyclic structure, hence a forest. By picking one of the clusters as the root, a parent-child relation is induced between clusters and articulation points. For QLDPC codes (bounded degree distributions), the Hopcroft-Tarjan algorithm runs in linear time on the size of the stopping set to build this cluster forest structure.

After the cluster decomposition is computed, the erasure decoding problem is split into several subproblems, associated to each cluster. In each cluster, a local solution must be computed, i.e., the variable nodes within the cluster must be assigned bits that match the syndrome in the check nodes within the cluster. Moreover, a consistency condition must be enforced on the cut nodes (articulation points) in order to combine solutions from neighboring clusters. The global solution is constructed by adopting a schedule to compute local solutions. Starting from the leaves of the forest, each leaf cluster solves its inner linear system through Gaussian elimination, and defines constraints on its parent cut node. These constraints are propagated by the cut node from its children to its own parent cluster node, and are in turn incorporated to the latter's internal linear system.

Since variable cut nodes require the local solutions from all neighboring clusters to agree on the value of that variable, clusters are classified as free or frozen, depending on whether local solutions exist among which the cut variable can take any assignment, or has is assignment enforced by the solutions of that cluster. The same classification is applied when a cluster is connected to a check cut node, by considering whether or not its contribution to the syndrome of that check node is determined by its local solutions. In the recursive construction of the global solution, the free/frozen classification determines whether a cut node or cluster propagates its constraints upwards. Frozen cut nodes of any type always propagate their constraints to the parent cluster, whereas free cut nodes don't. Frozen clusters propagate their constraints to the parent cut node, and its free/frozen is status depends on its type and on the status of its child clusters. A variable cut node is free when all its children are free, and otherwise the constraint propagated by its frozen children must agree. A check cut node on the other hand is frozen when all its children are frozen.

Once the process reaches the root cluster, one solution is computed, in compliance with the propagated constraints, and is then backpropagated in order to select one solution from each of the free clusters. At the end of this step, all neighboring clusters have local solutions consistent with each other, so they can be merged into a single global solution that is syndrome compliant. Notice that the algorithm always succeeds in outputting a solution, as is characteristic of the maximum-likelihood solution. In the backpropagation step, arbitrary choices are made to select solutions from the free clusters, which leads to the existence of multiple possible global solutions, only in the case where the erasure is effectively not correctable. Notice that by relaxing some of the consistency and propagation rules, the algorithm could be able to handle measurement errors.

In terms of complexity, since the computation of local solutions for each cluster is done via Gaussian elimination, this step alone has complexity of the order of the sum of the cubes of the cluster sizes, which is less than a cubic on the whole residual graph. Moreover, the cluster decomposition and propagation steps between the clusters adds only a linear time overhead. The complexity can be further controlled by adding an option to only compute local solutions for clusters of size up to some constant, declaring a decoding failure if any cluster is larger than that. This trades some of the decoding performance by an improvement in the runtime, a feature similar to the guessing budget in our algorithm. Final observation is that the performance of the size-constrained cluster decoder can be better understood by analyzing the distribution of the maximum cluster size for random residual graphs on a given code.

4.4.3 Degenerate quantum erasure decoding

In [28] the authors propose BP decoders that achieve near-capacity (in the erasure channel) performance for a range of codes, while also being adaptable to a mixture of erasure and Pauli noise.

4.5 Simulation results

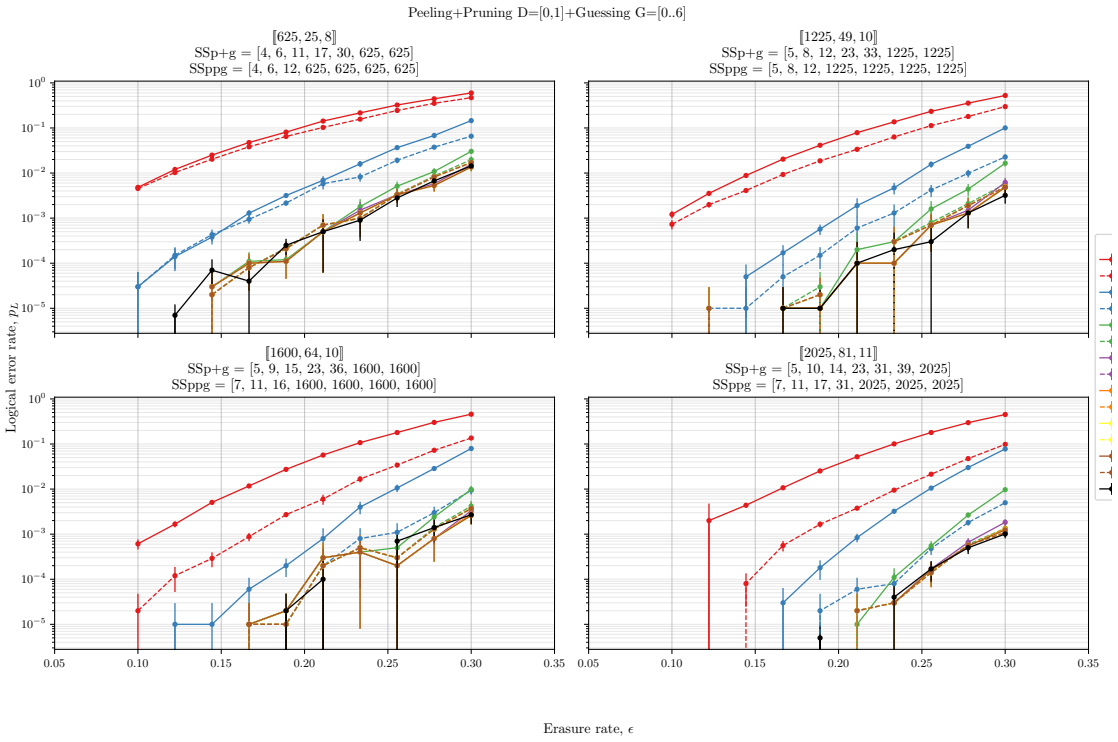


Figure 1: Performance of the PPG algorithm for some HGP codes

References

[1] Tom Richardson and Ruediger Urbanke. *Modern Coding Theory*. USA: Cambridge University Press, 2008. ISBN: 0521852293.

[2] M.G. Luby et al. "Efficient erasure correcting codes". In: *IEEE Transactions on Information Theory* 47.2 (2001), pp. 569–584. DOI: 10.1109/18.910575.

[3] Jean-Pierre Tillich and Gilles Zemor. "Quantum LDPC Codes With Positive Rate and Minimum Distance Proportional to the Square Root of the Blocklength". In: *IEEE Transactions on Information Theory* 60.2 (Feb. 2014), pp. 1193–1202. ISSN: 1557-9654. DOI: 10.1109/tit.2013.2292061. URL: http://dx.doi.org/10.1109/TIT.2013.2292061.

[4] Joschka Roffe et al. "Decoding across the quantum low-density parity-check code landscape". In: *Physical Review Research* 2.4 (Dec. 2020). ISSN: 2643-1564. DOI: 10.1103/physrevresearch.2.043423. URL: http://dx.doi.org/10.1103/PhysRevResearch.2.043423.

[5] Nicholas Connolly et al. "Fast erasure decoder for hypergraph product codes". In: *Quantum* 8 (Aug. 2024), p. 1450. ISSN: 2521-327X. DOI: 10.22331/q-2024-08-27-1450. URL: http://dx.doi.org/10.22331/q-2024-08-27-1450.

[6] Pavel Panteleev and Gleb Kalachev. *Asymptotically Good Quantum and Locally Testable Classical LDPC Codes*. 2022. arXiv: 2111.03654 [cs.IT]. URL: https://arxiv.org/abs/2111.03654.

[7] Anthony Leverrier and Gilles Zémor. *Quantum Tanner codes*. 2022. arXiv: 2202.13641 [quant-ph]. URL: https://arxiv.org/abs/2202.13641.

[8] Anthony Leverrier, Jean-Pierre Tillich, and Gilles Zemor. "Quantum Expander Codes". In: *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*. IEEE, Oct. 2015, pp. 810–824. DOI: 10.1109/focs.2015.55. URL: http://dx.doi.org/10.1109/FOCS.2015.55.

[9] Antoine Grospellier et al. "Combining hard and soft decoders for hypergraph product codes". In: *Quantum* 5 (Apr. 2021), p. 432. ISSN: 2521-327X. DOI: 10.22331/q-2021-04-15-432. URL: http://dx.doi.org/10.22331/q-2021-04-15-432.

[10] Nadine Meister, Christopher A. Pattison, and John Preskill. *Efficient soft-output decoders for the surface code*. 2024. arXiv: 2405.07433 [quant-ph]. URL: https://arxiv.org/abs/2405.07433.

[11] Eric Sabo et al. *Weight Reduced Stabilizer Codes with Lower Overhead*. 2024. arXiv: 2402.05228 [quant-ph]. URL: https://arxiv.org/abs/2402.05228.

[12] Thomas R. Scruby, Timo Hillmann, and Joschka Roffe. *High-threshold, low-overhead and single-shot decodable fault-tolerant quantum memory*. 2024. arXiv: 2406.14445 [quant-ph]. URL: https://arxiv.org/abs/2406.14445.

[13] Stasiu Wolanski and Ben Barber. *Ambiguity Clustering: an accurate and efficient decoder for qLDPC codes*. 2024. arXiv: 2406.14527 [quant-ph]. URL: https://arxiv.org/abs/2406.14527.

[14] David Poulin and Yeojin Chung. *On the iterative decoding of sparse quantum codes*. 2008. arXiv: 0801.1241 [quant-ph]. URL: https://arxiv.org/abs/0801.1241.

[15] Guanyu Zhu. *A topological theory for qLDPC: non-Clifford gates and magic state fountain on homological product codes with constant rate and beyond the $N^{1/3}$ distance barrier*. 2025. arXiv: 2501.19375 [quant-ph]. URL: https://arxiv.org/abs/2501.19375.

[16] Elena Berardini et al. *Asymptotically good CSS-T codes exist*. 2024. arXiv: 2412.08586 [quant-ph]. URL: https://arxiv.org/abs/2412.08586.

[17] Alexander J. Malcolm et al. *Computing Efficiently in QLDPC Codes*. 2025. arXiv: 2502.07150 [quant-ph]. URL: https://arxiv.org/abs/2502.07150.

[18] R. Tanner. "A recursive approach to low complexity codes". In: *IEEE Trans. Inf. Theor.* 27.5 (Sept. 2006), pp. 533–547. ISSN: 0018-9448. DOI: 10.1109/TIT.1981.1056404. URL: https://doi.org/10.1109/TIT.1981.1056404.

[19] Peter W. Shor. "Scheme for reducing decoherence in quantum computer memory". In: *Phys. Rev. A* 52 (4 Oct. 1995), R2493–R2496. DOI: 10.1103/PhysRevA.52.R2493. URL: https://link.aps.org/doi/10.1103/PhysRevA.52.R2493.

[20] Andrew M. Steane. "Error Correcting Codes in Quantum Theory." In: *Physical review letters* 77 5 (1996), pp. 793–797. URL: https://api.semanticscholar.org/CorpusID:35977016.

[21] Daniel Gottesman. *Stabilizer Codes and Quantum Error Correction*. 1997. arXiv: quant-ph/9705052 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9705052.

[22] Eric Dennis et al. "Topological quantum memory". In: *Journal of Mathematical Physics* 43.9 (Sept. 2002), pp. 4452–4505. ISSN: 1089-7658. DOI: 10.1063/1.1499754. URL: http://dx.doi.org/10.1063/1.1499754.

[23] Austin G. Fowler et al. "Surface codes: Towards practical large-scale quantum computation". In: *Physical Review A* 86.3 (Sept. 2012). ISSN: 1094-1622. DOI: 10.1103/physreva.86.032324. URL: http://dx.doi.org/10.1103/PhysRevA.86.032324.

[24] Oscar Higgott and Craig Gidney. *Sparse Blossom: correcting a million errors per core second with minimum-weight matching*. 2023. arXiv: 2303.15933 [quant-ph]. URL: https://arxiv.org/abs/2303.15933.

[25] Adway Patra and Alexander Barg. *Targeted Clifford logical gates for hypergraph product codes*. 2024. arXiv: 2411.17050 [quant-ph]. URL: https://arxiv.org/abs/2411.17050.

[26] Mert Gökduman, Hanwen Yao, and Henry D. Pfister. "Erasure Decoding for Quantum LDPC Codes via Belief Propagation with Guided Decimation". In: *2024 60th Annual Allerton Conference on Communication, Control, and Computing*. IEEE, Sept. 2024, pp. 1–8. DOI: 10.1109/allerton63246.2024.10735275. URL: http://dx.doi.org/10.1109/Allerton63246.2024.10735275.

[27] Hanwen Yao, Mert Gökduman, and Henry D. Pfister. "Cluster Decomposition for Improved Erasure Decoding of Quantum LDPC Codes". In: *IEEE Journal on Selected Areas in Information Theory* 6 (2025), pp. 176–188. DOI: 10.1109/JSAIT.2025.3578597.

[28] Kao-Yueh Kuo and Yingkai Ouyang. *Degenerate quantum erasure decoding*. 2025. arXiv: 2411.13509 [quant-ph]. URL: https://arxiv.org/abs/2411.13509.